

Estrutura de Dados
Projeto Interdisciplinar
Turma: 2NAADS
Professor: Eduardo Savino

GoMarket

Fátima Eduarda Gomes Balbino 25028242

Julia Gomes Basílio 25028157

Maria Eduarda Barberino Olo 25028088

Luis Felipe Trindade 22076554

São Paulo

2026

1. Introdução

Este relatório descreve as estruturas de dados e estratégias algorítmicas aplicadas no desenvolvimento da plataforma GoMarket, um marketplace B2B de anúncios de alimentos.

O sistema implementa operações CRUD completas (Create, Read, Update, Delete) com autenticação via JWT, integrando um back-end em Node.js/Express com banco de dados MySQL e um front-end em JavaScript puro.

O objetivo deste documento é demonstrar como o conhecimento de estruturas de dados foi aplicado para otimizar buscas, ordenações e o controle de sessão na aplicação.

2. Visão Geral da Arquitetura

A aplicação segue o padrão de três camadas (3-tier):

- **Frontend (SPA):** JavaScript (ES6), HTML e CSS
- **Backend (API REST):** Node.js com Express
- **Banco de Dados:** MySQL com pool de conexões

Essa separação melhora a organização, manutenção e escalabilidade do sistema.

3. Estruturas de Dados Aplicadas

3.1 Set (Conjunto) — Denylist de Tokens JWT

No serviço de autenticação (tokenService.js), utiliza-se a estrutura nativa Set do JavaScript para implementar uma denylist de tokens revogados:

```
const denylist = new Set()
export const denyToken = (jti) => { if (jti) denylist.add(jti) }
export const isDenied = (jti) => denylist.has(jti)
```

Análise de complexidade:

Operação	Estrutura	Complexidade
Revogar token (logout)	Set.add(jti)	O(1) amortizado
Verificar token revogado	Set.has(jti)	O(1)
Array simples (alternativa)	Array.includes(jti)	O(n)

A escolha do Set em vez de um Array garante verificação de revogação em tempo constante O(1), independentemente da quantidade de tokens revogados. Com um Array, cada requisição

exigiria percorrer todos os elementos — $O(n)$ —, degradando a performance em cenários de alto volume de logouts.

3.2 Pool de Conexões — Fila de Recursos

O módulo `db.js` utiliza um pool de conexões do `mysql2`, que internamente implementa uma fila (Queue) de conexões disponíveis:

```
export const pool = await mysql.createPool({
  connectionLimit: 10,
  waitForConnections: true,
  ...
})
```

O pool evita o custo de $O(1)$ de abertura/fechamento de conexão por requisição (que na prática é lento por envolver handshake TCP e autenticação MySQL). Conexões são reutilizadas em FIFO, mantendo até 10 simultâneas e enfileirando requisições excedentes em vez de falhar.

3.3 Hash Map Implícito — Índice UNIQUE no Banco

A tabela `users` define o campo `email` como `UNIQUE INDEX` no MySQL:

```
email VARCHAR(150) NOT NULL UNIQUE
```

Internamente, o MySQL implementa esse índice como uma estrutura de B-Tree (árvore balanceada), que fornece busca por email em $O(\log n)$. A constraint `UNIQUE` também é verificada via hash em $O(1)$ durante `INSERTs`, garantindo que a aplicação não precise fazer `SELECT` antes de inserir — o próprio banco retorna o erro `ER_DUP_ENTRY`, tratado explicitamente nos controllers.

3.4 Array de Registros — Listagem de Usuários

A consulta de listagem no back-end utiliza `ORDER BY id DESC`, delegando a ordenação ao motor do MySQL:

```
SELECT id, name, email, img, created_at FROM users ORDER BY id DESC
```

O campo `id` é chave primária (índice clustered B-Tree). A ordenação `DESC` aproveita a travessia reversa da árvore em $O(n)$, sem criar estrutura adicional. No front-end, o retorno é armazenado

em um Array JavaScript e renderizado via `map()` — $O(n)$ — para geração das linhas da tabela HTML.

3.5 Hashing — Armazenamento Seguro de Senhas

As senhas são armazenadas com `bcrypt` (fator de custo 10):

```
const hash = await bcrypt.hash(password, 10) // O(2^10) operações
const ok   = await bcrypt.compare(plain, hash) // O(2^10) operações
```

O `bcrypt` aplica uma função de hash não-reversível com sal aleatório, tornando ataques de dicionário e rainbow tables inviáveis. A complexidade $O(2^{\text{cost}})$ é intencional — dificulta força bruta — mas é executada apenas em login e cadastro, sem impacto nas operações frequentes de leitura.

4. Buscas Aplicadas

Operação	Onde	Algoritmo / Estrutura	Complexidade
Buscar por email (login)	MySQL — <code>authController</code>	Índice B-Tree no campo email	$O(\log n)$
Buscar por ID	MySQL — <code>useController</code>	PK clustered (B-Tree)	$O(\log n)$
Verificar token revogado	<code>tokenService</code> — <code>Set.has()</code>	Tabela hash interna do Set	$O(1)$
Buscar elemento no DOM	Front-end — <code>app.js</code>	<code>querySelector</code> / <code>closest()</code>	$O(n)$ DOM
Verificar email duplicado	MySQL — INSERT UNIQUE	Hash index	$O(1)$

Destaque: a busca por email no login não utiliza `SELECT * FROM users` (varredura completa $O(n)$), mas sim `WHERE email = ?` que aciona o índice UNIQUE e obtém complexidade $O(\log n)$ via B-Tree.

5. Ordenações Aplicadas

Contexto	Critério	Implementação	Complexidade
Listagem de usuários (back-end)	id DESC (mais recentes primeiro)	ORDER BY id DESC — MySQL B-Tree reverso	$O(n)$
Renderização no front-end	Mantém ordem do servidor	Array.map() — sem reordenação adicional	$O(n)$
Tokens na denylist	Sem ordem (lookup puro)	Set (hash table)	$O(1)$ por operação

A ordenação foi delegada ao banco de dados, que é otimizado para isso. Ordenar no JavaScript (Array.sort — $O(n \log n)$) seria desnecessário e menos eficiente do que usar o índice primário já existente.

6. Como o Conhecimento de Estrutura de Dados Otimizou a Aplicação

Problema	Solução com ED	Ganho
Verificar logout sem consultar BD	Set em memória ($O(1)$)	Elimina round-trip ao banco por requisição
Evitar N+1 queries na listagem	Único SELECT com ORDER BY	1 query para n usuários
Unicidade de email	UNIQUE INDEX (B-Tree)	Verificação no banco sem SELECT prévio
Gerenciar conexões simultâneas	Pool como fila (Queue)	Reutilização de conexões, sem overhead TCP
Segurança de senhas	Hash bcrypt com sal	Resistência a rainbow table e força bruta
Roteamento SPA sem reload	Array de views (data-view)	Troca $O(k)$ de seções sem request HTTP

7. Conclusão

O projeto demonstra aplicação consciente de estruturas de dados em diferentes camadas da aplicação. A escolha do Set para a denylist de tokens elimina consultas ao banco de dados no fluxo de autenticação, reduzindo latência. O uso de índices B-Tree no MySQL garante buscas e ordenações eficientes. O pool de conexões implementa uma fila que maximiza o reuso de recursos. No front-end, a arquitetura de roteamento baseada em Arrays de nós DOM evita recarregamentos desnecessários.

Cada decisão foi orientada pela análise de complexidade de tempo e espaço, priorizando operações $O(1)$ e $O(\log n)$ nos pontos críticos do fluxo de dados.